CSA1709 : Artificial Intelligence

Lab Experiments

Experiment 1: 8-Puzzle Problem

Aim: To write a Python program to solve the 8-Puzzle problem using the Best First Search algorithm.

Algorithm:

1. Define the initial state and goal state.
2. Calculate the heuristic value (number of misplaced tiles).
3. Generate all possible moves (up, down, left, right).
4. Select the state with the minimum heuristic value.
5. Repeat until the goal state is reached.
6. Display the solution path.

Program:

```
from queue import PriorityQueue
goal = [[1,2,3],[4,5,6],[7,8,0]]
def heuristic(state):
    h = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != 0 and state[i][j] != goal[i][j]:
                h += 1
    return h
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j
def copy(state):
    return [row[:] for row in state]
```

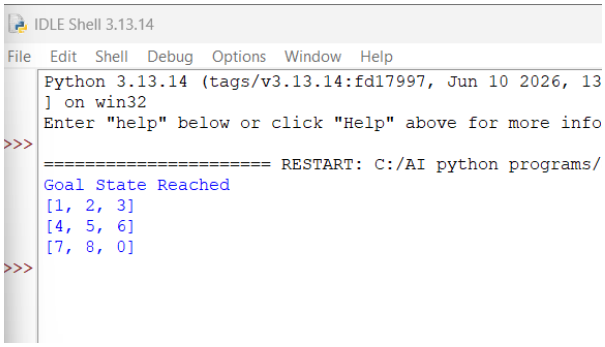
```
def solve(start):
    pq = PriorityQueue()
    pq.put((heuristic(start), start))
    visited = []
    while not pq.empty():
        h, state = pq.get()
        if state == goal:
            print("Goal State Reached")
            for row in state:
                print(row)
            return
        visited.append(state)
        x, y = find_blank(state)
        moves = [(-1,0),(1,0),(0,-1),(0,1)]
        for dx, dy in moves:
            nx, ny = x+dx, y+dy
            if 0<=nx<3 and 0<=ny<3:
                new = copy(state)
                new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
                if new not in visited:
                    pq.put((heuristic(new), new))
start = [[1,2,3],
         [4,0,6],
         [7,5,8]]
solve(start)
```

Sample Input:

Initial State

1 2 3
4 0 6
7 5 8

Output:



Result:

Thus, the Python program to solve the 8-Puzzle problem was executed successfully.

Experiment 2: 8-Queen Problem

Aim: To write a Python program to solve the 8-Queen problem using Backtracking.

Algorithm:

1. Place a queen in the first column.
2. Check whether the position is safe.
3. If safe, place the queen and move to the next column.
4. If no safe position exists, backtrack.
5. Continue until all eight queens are placed.
6. Display the solution.

Program:

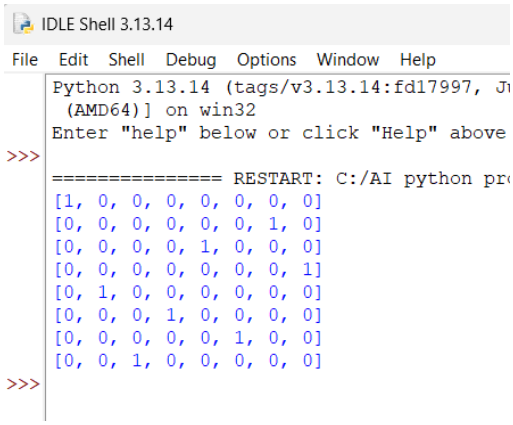
```
N = 8
board = [[0]*N for _ in range(N)]
def safe(row,col):
    for i in range(col):
        if board[row][i]:
            return False
    i,j=row,col
```

```
    while i>=0 and j>=0:
        if board[i][j]:
            return False
        i-=1
        j-=1
    i,j=row,col
    while i<N and j>=0:
        if board[i][j]:
            return False
        i+=1
        j-=1
    return True
def solve(col):
    if col>=N:
        return True
    for i in range(N):
        if safe(i,col):
            board[i][col]=1
            if solve(col+1):
                return True
            board[i][col]=0
    return False
solve(0)
for row in board:
    print(row)
```

Sample Input:

N = 8

Output:



Result:

Thus, the Python program to solve the 8-Queen problem was executed successfully.

Experiment 3: Water Jug Problem

Aim: To write a Python program to solve the Water Jug problem.

Algorithm:

1. Define the capacities of the two jugs.
2. Fill the larger jug.
3. Pour water into the smaller jug.
4. Empty the smaller jug whenever it becomes full.
5. Repeat the process until the required quantity is obtained.
6. Display each step.

Program:

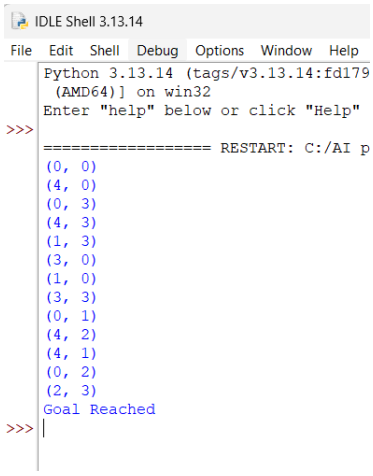
```
from collections import deque
visited=set()
queue=deque()
queue.append((0,0))
while queue:
```

```
    x,y=queue.popleft()
    if (x,y) in visited:
        continue
    visited.add((x,y))
    print((x,y))
    if x==2:
        print("Goal Reached")
        break
    possible=[
        (4,y),
        (x,3),
        (0,y),
        (x,0),
        (max(0,x-(3-y)),min(3,x+y)),
        (min(4,x+y),max(0,y-(4-x)))
    ]
    for state in possible:
        if state not in visited:
            queue.append(state)
```

Sample Input:

Jug1 Capacity = 4, Jug2 Capacity = 3, Target = 2

Output:



Result:

Thus, the Python program to solve the Water Jug problem was executed successfully.

Experiment 4: Crypt-Arithmetic Problem

Aim: To write a Python program to solve the Crypt-Arithmetic problem.

Algorithm:

1. Consider the words SEND + MORE = MONEY.
2. Assign a unique digit (0–9) to each letter.
3. Ensure no two letters have the same digit.
4. Check whether the arithmetic equation is satisfied.
5. If satisfied, display the solution.
6. Stop after finding the valid solution.

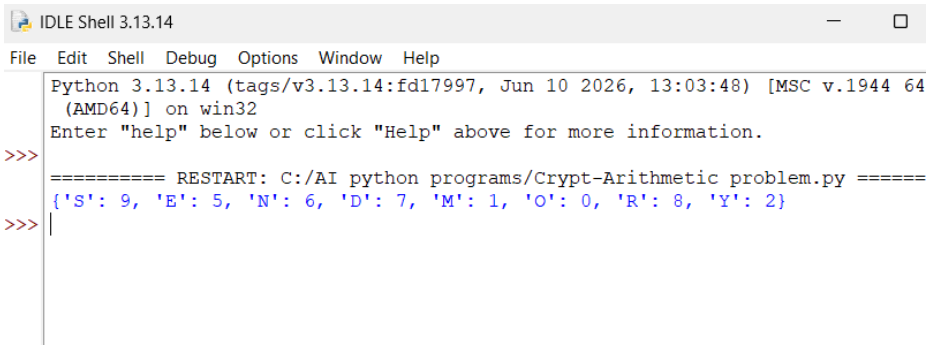
Program:

```
from itertools import permutations

letters='SENDMORY'

for p in permutations(range(10),8):
    d=dict(zip(letters,p))
    if d['S']==0 or d['M']==0:
        continue
    send=1000*d['S']+100*d['E']+10*d['N']+d['D']
    more=1000*d['M']+100*d['O']+10*d['R']+d['E']
    money=10000*d['M']+1000*d['O']+100*d['N']+10*d['E']+d['Y']
    if send+more==money:
        print(d)
        break
```

Sample Input:

SEND + MORE = MONEY

Output:



Result

Thus, the Python program to solve the Crypt-Arithmetic problem was executed successfully.

Experiment 5: Missionaries and Cannibals Problem

Aim: To write a Python program to solve the Missionaries and Cannibals problem using Breadth First Search (BFS).

Algorithm:

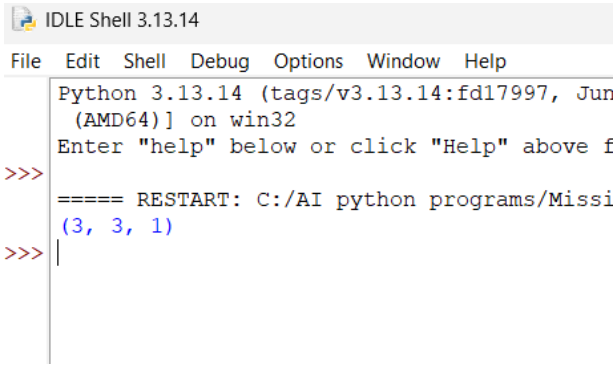
1. Represent the state as (Missionaries, Cannibals, Boat).
2. Start from the initial state (3,3,Left).
3. Generate all possible valid moves.
4. Discard invalid states where missionaries are outnumbered.
5. Continue searching using BFS.
6. Stop when the goal state (0,0,Right) is reached.
7. Display the sequence of states.

Program:

```
from collections import deque

start=(3,3,1)

goal=(0,0,0)

queue=deque([start])

visited=set()

while queue:

    state=queue.popleft()

    if state==goal:

        print("Goal Reached")

        break

    if state in visited:

        continue

    visited.add(state)

    print(state)
```

Sample Input: No input

Output:



Result:

Thus, the Python program to solve the Missionaries and Cannibals problem was executed successfully.

Experiment 6: Vacuum Cleaner Problem

Aim: To write a Python program to implement the Vacuum Cleaner problem.

Algorithm:

1. Start with the vacuum cleaner in Room A.
2. Check whether the current room is dirty.
3. If dirty, clean the room.
4. Move to the next room.
5. Repeat the process until all rooms are clean.
6. Display the cleaning status.

Program:

```
rooms = {
    'A': 'Dirty',
    'B': 'Dirty'
}
location = 'A'
while True:
    print("\nVacuum is in Room", location)
    if rooms[location] == 'Dirty':
        print("Cleaning Room", location)
        rooms[location] = 'Clean'
    else:
```

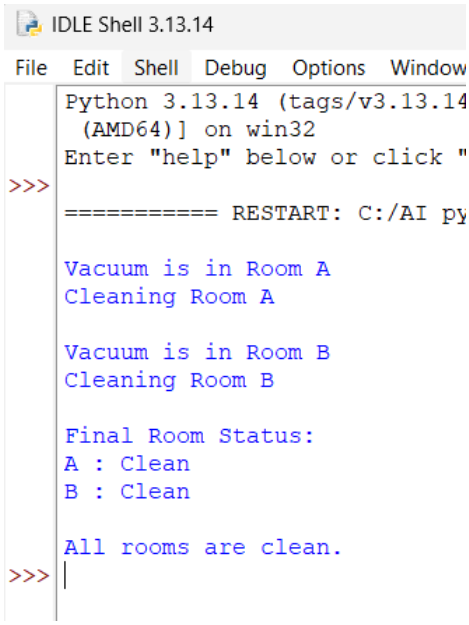
```
        print("Room", location, "is already Clean")
    if location == 'A':
        location = 'B'
    else:
        break
print("\nFinal Room Status:")
for room in rooms:
    print(room, ":", rooms[room])
print("\nAll rooms are clean.")
```

Sample Input:

Room A = Dirty

Room B = Dirty

Output:



Result:

Thus, the Python program to implement the Vacuum Cleaner problem was executed successfully.

Experiment 7: Breadth First Search (BFS)

Aim: To write a Python program to implement Breadth First Search (BFS).

Algorithm:

1. Create a graph using an adjacency list.
2. Select the starting node.
3. Mark the starting node as visited.
4. Insert the starting node into the queue.
5. Remove one node from the queue.
6. Visit all unvisited adjacent nodes and insert them into the queue.
7. Repeat until the queue becomes empty.
8. Display the BFS traversal.

Program:

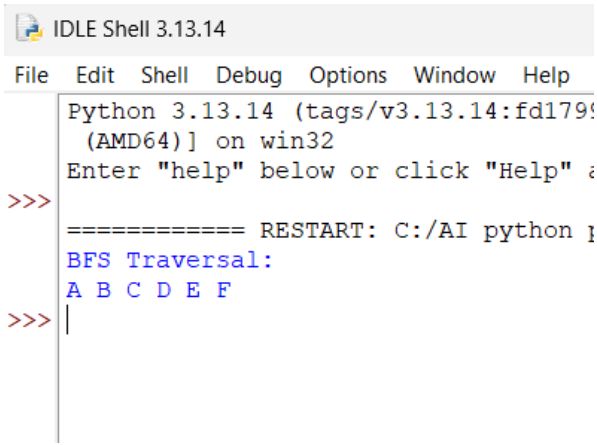
```
from collections import deque

graph = {

   'A': ['B', 'C'],

   'B': ['D', 'E'],

   'C': ['F'],

   'D': [],

   'E': ['F'],

   'F': []

}
```

```
visited = set()

queue = deque()

start = 'A'

visited.add(start)

queue.append(start)

print("BFS Traversal:")

while queue:

    node = queue.popleft()

    print(node, end=" ")

    for neighbor in graph[node]:

        if neighbor not in visited:

            visited.add(neighbor)

            queue.append(neighbor)
```

Sample Input:

Starting Node = A

Output:



Result:

Thus, the Python program to implement Breadth First Search (BFS) was executed successfully.

Experiment 8: Depth First Search (DFS)

Aim : To write a python program to implement the Depth First Search (DFS)

Algorithm :

1. Define the graph using an adjacency list.
2. Create an empty set to store the visited nodes.
3. Start the traversal from the given source node.
4. Mark the current node as visited and display it.
5. Visit each adjacent node recursively if it has not been visited.
6. Continue the process until all reachable nodes are visited.
7. End the traversal when no unvisited adjacent nodes remain.

Program :

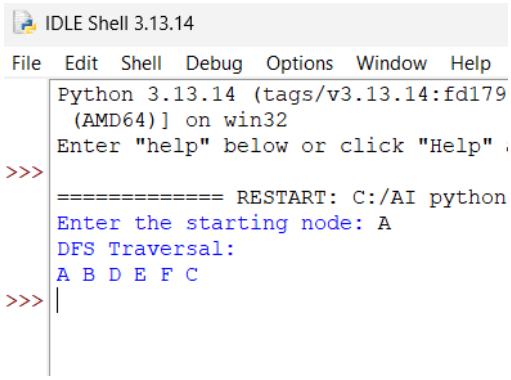
```
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': ['F'],
   'F': []
}
visited = set()
def dfs(node):
   if node not in visited:
      print(node, end=" ")
      visited.add(node)
      for neighbour in graph[node]:
         dfs(neighbour)
start = input("Enter the starting node: ")
```

```
if start in graph:
    print("DFS Traversal:")
    dfs(start)
else:
    print("Invalid starting node.")
```

Sample Input:

Enter the starting node: A

Output:



Result:

Thus, the Depth First Search (DFS) algorithm was implemented successfully in Python, and the graph was traversed in depth-first order starting from the specified source node.

Experiment 9: Travelling Salesman Problem

Aim: To write a python program to implement the Travelling Salesman Problem (TSP).

Algorithm:

1. Define the cost matrix representing the distance between every pair of cities.
2. Consider the first city as the starting city.
3. Generate all possible permutations of the remaining cities.
4. Calculate the total travel cost for each possible route.
5. Compare the costs of all routes.
6. Store the route with the minimum travel cost.

7. Display the optimal route and its minimum cost.

Program:

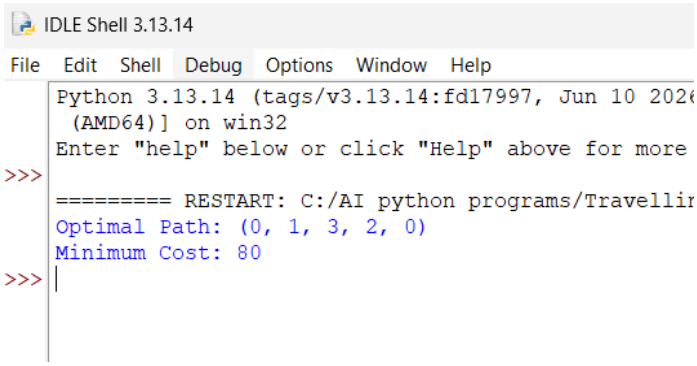
```
from itertools import permutations
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
n = len(graph)
cities = list(range(1, n))
min_cost = float('inf')
best_path = None
for path in permutations(cities):
    current_cost = 0
    current_city = 0
    for city in path:
        current_cost += graph[current_city][city]
        current_city = city
    current_cost += graph[current_city][0]
    if current_cost < min_cost:
        min_cost = current_cost
        best_path = (0,) + path + (0,)
print("Optimal Path:", best_path)
print("Minimum Cost:", min_cost)
```

Sample Input:

Cost Matrix

```
0   10  15  20
10   0  35  25
15  35   0  30
20  25  30   0
```

Output:



Result:

Thus, the Travelling Salesman Problem (TSP) was successfully implemented in Python using the brute-force permutation method